



NEAR Intents TEE Solver Registry and AMM Solver

Security Assessment

January 19, 2025

Prepared for:

Robert Yan

NEAR One

Prepared by: **Samuel Moelius**

Table of Contents

Table of Contents	1
Project Summary	2
Executive Summary	3
Project Goals	6
Project Targets	7
Project Coverage	8
Summary of Findings	9
Detailed Findings	10
1. Lack of two-step process for ownership transfers	10
2. Code duplication	12
3. Insufficient test coverage	13
4. Invalid data returned from dstack SDK would cause AMM Solver to crash	14
5. Lack of an mt_batch_balance_of method causes AMM Solver to crash	16
6. Race condition in create_liquidity_pool	18
A. Vulnerability Categories	21
B. Non-Security-Related Recommendations	23
C. Steps to Reproduce TOB-TEESOLVER-4	26
D. Fix Review Results	27
Detailed Fix Review Results	28
E. Fix Review Status Categories	31
About Trail of Bits	32
Notices and Remarks	33

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultant was associated with this project:

Samuel Moelius, Consultant
samuel.moelius@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
August 26, 2025	Pre-project kickoff call
September 8, 2025	Status update meeting #1
September 15, 2025	Delivery of report draft
September 15, 2025	Report readout meeting
September 16, 2025	Delivery of final comprehensive report
January 15, 2026	Completion of fix review
January 19, 2025	Addition of Appendix D: Fix Review Results

Executive Summary

Engagement Overview

NEAR One engaged Trail of Bits to review the security of its TEE Solver Registry and AMM Solver:

- The TEE Solver Registry is a repository consisting of two NEAR contracts. The Solver Registry contract holds details about workers that will run Docker compose files with pre-registered hashes in TDX enclaves. The other NEAR contract is the Intents Vault, which holds the workers' public keys.
- The AMM Solver is an example of a worker that will run in a TDX enclave. Specifically, the worker is intended to run on [Phala Cloud](#).

One consultant conducted the review from September 2 to September 12, 2025, for a total of two engineer-weeks of effort. Our testing efforts focused on ways that attestation might be misimplemented. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes.

Observations and Impact

The code is lacking in automated testing:

- The Solver Registry contract has a substantial test suite. However, many parts of the contract's code are not exercised by any test ([TOB-TEESOLVER-3](#)). Furthermore, many tests simply check whether an operation succeeded or failed. Tests that check for failure should check that the operation failed for the expected reason ([appendix B](#)).
- The Intents Vault contract has no direct unit tests. Having even a few simple unit tests would help prevent the contract's behavior from being accidentally changed by a future commit.
- The AMM Solver has exactly two tests to check the correctness of token quotes. The AMM Solver has no tests of its interaction with the NEAR network, proper web server operation, handling of configuration files, logging, or caching, for example.

To add to the last bullet point, we spent considerable time ensuring that we could deploy the AMM Solver to Phala Cloud. During this investigation, it was determined that the existing Docker image did not attach sufficient gas to the `register_worker` call and that a [fix](#) was needed, which Near One has already implemented. Problems like this would likely be exposed by more thorough automated testing.

There are many situations where the code terminates without providing a useful error message. For example, if invalid data were returned from the [dstack SDK](#), the AMM Solver

would crash (TOB-TEESOLVER-4). Separately, if the AMM solver is configured with an intents contract that lacks an `mt_batch_balance_of` method, it will crash (TOB-TEESOLVER-5). Finally, the AMM Solver does not validate that the network and the named accounts it is configured to use coincide. This can cause the AMM Solver to fail to start with inaccurate “account...doesn’t exist” errors that are likely to confuse users (appendix B).

The code has at least one confirmed race condition (TOB-TEESOLVER-6).¹ The race condition manifests itself when multiple users try to call `create_liquidity_pool` at the same time. In some cases, both calls can appear to succeed when, in reality, only one call succeeds. We suspect (though we were unable to confirm) that similar bugs may exist in `deposit_into_pool` and `register_new_public_key`.

Overall, we found the code to be well structured and easy to read. However, in addition to the preceding points, the code would benefit from comments describing the purpose of each function.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that NEAR One take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Expand the Solver Registry’s, Intents Vault’s, and AMM Solver’s tests.** The projects’ conventional tests can all be expanded. The projects would also benefit from more advanced testing techniques, such as fuzzing or property testing.
- **Improve the AMM Solver’s error reporting.** If the AMM Solver cannot start or must shut down, ensure it emits an informative error message. Doing so will ease users’ error reporting and the diagnosis of the problems the errors describe.
- **Investigate the TEE Solver’s use of callbacks and whether they introduce race conditions.** The TEE Solver contains at least one race condition in its use of callbacks in `create_liquidity_pool`. We suspect similar race conditions may exist in `deposit_into_pool` and `register_new_public_key`. We recommend that the functions’ callbacks panic when the passed-in callback result is an error. Doing so will communicate to the environment that the call has failed.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

¹ This line of investigation was inspired by “Combating Reentrancy Bugs on Sharded Blockchains” by Ognjen Maric, Robin Kunzler, Lara Schmid, and Roman Kashitsyn.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	0
Medium	0
Low	3
Informational	3
Undetermined	0

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Access Controls	1
Error Reporting	2
Patching	1
Testing	1
Timing	1

Project Goals

The engagement was scoped to provide a security assessment of the NEAR One TEE Solver Registry and AMM Solver. Specifically, we sought to answer the following non-exhaustive list of questions:

- Are there any ways assets can be stolen?
- Can a party's private keys be leaked through the use of the platform?
- Can an unauthorized user upgrade the Solver Registry with malicious code?
- Is it possible to bypass the Intents Vault's parent account check?
- Can a worker register with a Docker compose hash that has not been approved?

Project Targets

The engagement involved reviewing and testing the following targets.

NEAR Intents TEE Solver Registry

Repository	https://github.com/Near-One/tee-solver
Version	9340697221145f27c868693f423a6c936226d72b aad71b44b8a9c7ae045797e44cd1f6333776de62 ca3913abd67662190dc34638f8677cbac083ca0f
Type	Rust
Platform	NEAR

AMM Solver

Repository	https://github.com/think-in-universe/near-intents-tee-amm-solver
Version	bea9a415e550a724d8b5fe40fa214fe46265f926
Branch	feat/tee-solver
Type	TypeScript
Platform	Phala Cloud

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals.

For the Rust code, our approaches included the following:

- **Dependency review.** We ran `cargo-audit` to determine whether the TEE Solver Registry relies on any vulnerable dependencies.
- **Test coverage review.** We ran the Intents Vault and Solver Registry tests to verify that they pass. We also reviewed the tests to look for important conditions that might not be tested.
- **Static analysis.** We ran `Clippy` over the source code and reviewed the results.
- **Manual review.** We manually reviewed the Intents Vault and Solver Registry source code.

For the TypeScript code, our approaches included the following:

- **Test coverage review.** We ran the AMM Solver tests to verify that they pass.
- **Static analysis.** We ran `Semgrep` and `CodeQL` over the source code and reviewed the results.
- **Manual review.** We manually reviewed the AMM Solver source code.
- **Dynamic testing.** We ran the AMM Solver locally and tested it with different configurations. We also deployed it to Phala Cloud to verify that this was possible.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- While we did test the AMM Solver with different configurations (e.g., `NETWORK_ID`, `NEAR_NODE_URLS`, etc.), we did not attempt to interact with it once it was running.
- We did not test the AMM Solver on TDX-enabled hardware beyond deploying it to Phala Cloud.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Lack of two-step process for ownership transfers	Access Controls	Informational
2	Code duplication	Patching	Informational
3	Insufficient test coverage	Testing	Informational
4	Invalid data returned from dstack SDK would cause AMM Solver to crash	Error Reporting	Low
5	Lack of an mt_batch_balance_of method causes AMM Solver to crash	Error Reporting	Low
6	Race condition in create_liquidity_pool	Timing	Low

Detailed Findings

1. Lack of two-step process for ownership transfers

Severity: Informational

Difficulty: High

Type: Access Controls

Finding ID: TOB-TEESOLVER-1

Target: contracts/solver-registry/src/admin.rs

Description

When called, the `change_owner` method immediately sets the contract owner to the provided account ID. The use of a single step to make such a critical change is error-prone; if the function is called with erroneous input, the results could be irrevocable or difficult to recover from. Requiring a separate acceptance step before role transfer is completed would guarantee that the address of the new owner contract is able to properly execute transactions on the contracts it will be administering.

```
35     pub fn change_owner(&mut self, new_owner_id: AccountId) {
36         self.assert_owner();
37         let old_owner_id = self.owner_id.clone();
38         self.owner_id = new_owner_id.clone();
39
40         Event::OwnerChanged {
41             old_owner_id: &old_owner_id,
42             new_owner_id: &new_owner_id,
43         }
44         .emit();
45     }
```

*Figure 1.1: Definition of the Solver Registry's `change_owner` method
([tee-solver/contracts/solver-registry/src/admin.rs#35-45](#))*

Exploit Scenario

Alice invokes `change_owner` to change the contract owner but accidentally enters the wrong address. As a result, she loses the ability to approve and remove Docker compose hashes.

Recommendations

Short term, implement a two-step process for all irrecoverable critical operations. Consider splitting the `change_owner` method into two steps: one to propose the new owner, and another in which the new owner accepts ownership of the contract.

Long term, identify and document all possible actions that can be taken by privileged accounts, along with their associated risks. This will facilitate review of the codebase and help prevent future mistakes.

2. Code duplication

Severity: Informational

Difficulty: High

Type: Patching

Finding ID: TOB-TEESOLVER-2

Target: contracts/solver-registry/src/attestation subdirectory

Description

The Solver Registry contract openly copies attestation code from the **NEAR MPC** project (figure 2.1). A risk associated with duplicated code is that a bug could be fixed in one place, but still reside in another.

```
1  ## TEE Attestation
2
3  The current TEE attestation module reuses the implementation from [NEAR
4  MPC](https://github.com/near/mpc) TEE attestation:
5  - Referenced source code on 2025-09-03:
6  https://github.com/near/mpc/tree/4241e195016c6667ec4d796399759bea1060ef1a/crates/attestation/src
```

*Figure 2.1: Message in the attestation subdirectory's README
(tee-solver/contracts/solver-registry/src/attestation/README.md#1-4)*

Exploit Scenario

Alice, a NEAR MPC developer, is tasked with fixing a bug in the project's attestation code. Alice does so, but fails to notify the Solver Registry contract developers. The bug persists in the Solver Registry contract's attestation code.

Recommendations

Short term, pursue one of the following options:

- Convince the NEAR MPC developers to publish their attestation code to **crates.io**. Use the crate from crates.io so that copying the attestation code is unnecessary.
- Refer to the attestation code as a **Git dependency**.

Note that either of these options may require changes to NEAR MPC's attestation code. However, using the NEAR MPC code directly will reduce the likelihood that a bug is fixed there and the fix is not propagated to the Solver Registry's code.

Long term, as new code is added to the codebase, resist the urge to copy existing code. Doing so will result in more maintainable code that is less likely to contain bugs.

3. Insufficient test coverage

Severity: Informational

Difficulty: High

Type: Testing

Finding ID: TOB-TEESOLVER-3

Target: contracts/solver-registry contract

Description

Several parts of the Solver Registry code are not executed by any test. Without test coverage, it is impossible to determine whether code is operating correctly.

The following parts of the Solver Registry code are not executed by any test:

- Function `remove_compose_hash`
- Function `change_owner`
- Error case in `find_approved_compose_hash`
- Error case in `on_create_liquidity_pool_account`
- Function `on_deposit_into_pool`
- Function `deposit_into_pool`
- Function `ft_on_transfer`
- Function `migrate`
- Function `upgrade`
- Function `get_owner_id`
- Function `get_approved_compose_hash`
- Function `get_pool_len`
- Function `get_worker_len`
- Function `get_workers`
- Function `get_worker_ping_timeout`

Exploit Scenario

A bug is discovered in the Solver Registry code. The bug might have been uncovered through more thorough testing.

Recommendations

Short term, add tests so that each of the items in the above bulleted list is tested. Taking these steps will help to expose bugs in the codebase.

Long term, regularly review the project's test coverage. If critical functions or components are untested, invest time in expanding the project's tests.

4. Invalid data returned from dstack SDK would cause AMM Solver to crash

Severity: Low

Difficulty: High

Type: Error Reporting

Finding ID: TOB-TEESOLVER-4

Target: `src/utils/agent.ts`

Description

Testing the AMM Solver revealed that if `DstackClient::getQuote` were to return invalid data, the AMM Solver would crash. To prevent the problem, additional checks should be added to the AMM Solver.²

To be more specific, we tested the AMM Solver with a version of dstack whose `DstackClient::getQuote` function returned the object in figure 4.1. This resulted in the error message in figure 4.2. Note that the object does not conform to the `GetQuoteResponse` interface (figure 4.3).

```
{"detail": "Method Not Allowed"}
```

Figure 4.1: Object returned by `DstackClient::getQuote`

```
.../src/utils/agent.ts:134
  const quote_hex = ra.quote.replace(/^0x/, '');
                                ^
TypeError: Cannot read properties of undefined (reading 'replace')
```

Figure 4.2: Error message produced by the AMM Solver

```
62 export interface GetQuoteResponse {
63   quote: Hex
64   event_log: string
65
66   replayRtmrs: () => string[]
67 }
```

Figure 4.3: `GetQuoteResponse` interface (`dstack/sdk/js/src/index.ts#62-67`)

Steps to reproduce this finding appear in [appendix C](#).

² An earlier version of this finding claimed there was a problem in the dstack SDK. However, NEAR One has since shown us that we were using the wrong version of dstack.

Exploit Scenario

Alice tries to run an AMM Solver. Invalid data returned from the dstack SDK causes it to crash. The error message in figure 4.2 does not help Alice determine the cause. Alice wastes time and energy looking for a bug in the AMM Solver.

Recommendations

Short term, add checks to verify that data returned from the dstack SDK conforms to its stated interface. Doing so will help prevent errors that can cause the AMM Solver to crash.

Long term, adopt a more defensive programming style. When interacting with external services, assume that returned data will be malformed and must be checked for validity. Adopting such a strategy will help prevent bugs like the one described here.

5. Lack of an `mt_batch_balance_of` method causes AMM Solver to crash

Severity: Low

Difficulty: Medium

Type: Error Reporting

Finding ID: TOB-TEESOLVER-5

Target: `src/services/intents.service.ts`

Description

To determine whether a worker is ready, the AMM Solver's `IntentsService` calls `mt_batch_balance_of` on the configured intents contract (figure 5.1). However, if the configured contract does not have an `mt_batch_balance_of` method, the AMM Solver crashes with an unhelpful error message (figure 5.2). The `IntentsService` should include checks to guard against this possibility.

```
19 public async getBalancesOnContract(tokenIds: string[]) {
20   const account = this.nearService.getAccount();
21   const result = await account.viewFunction({
22     contractId: intentsContract,
23     methodName: 'mt_batch_balance_of',
24     args: {
25       account_id: this.nearService.getIntentsAccountId(),
26       token_ids: tokenIds,
27     },
28   });
29   ...
34 }
```

Figure 5.1: Call to `mt_batch_balance_of` in `IntentsService`
([near-intents-tee-amm-solver/src/services/intents.service.ts#19-34](#))

```
.../node_modules/@near-js/providers/lib/json-rpc-provider.js:406
      throw new utils_1.ServerError((0,
utils_1.formatError)(errorType, params), errorType);
      ^
ServerError: Contract method is not found
    at JsonRpcProvider.<anonymous>
    (.../node_modules/@near-js/providers/lib/json-rpc-provider.js:406:35)
    at Generator.next (<anonymous>)
    at fulfilled (.../node_modules/@near-js/providers/lib/json-rpc-provider.js:5:58)
    at processTicksAndRejections (node:internal/process/task_queues:95:5) {
  type: 'MethodNotFound',
  context: undefined
}
```

Figure 5.2: Error message returned when `mt_batch_balance_of` method is not found

Exploit Scenario

Alice tries to run an AMM Solver. However, she configures it with an intents contract that does not have an `mt_batch_balance_of` method. The AMM Solver crashes, but Alice is unable to determine the cause from the error message.

Recommendations

Short term, add checks to verify that the call to `mt_batch_balance_of` does not result in "Contract method is not found." Doing so will help prevent errors that can cause the AMM Solver to crash.

Long term, adopt a more defensive programming style. When interacting with external services, assume that returned data will be malformed and must be checked for validity. Adopting such a strategy will help prevent bugs like the one described here.³

³ The same long-term recommendation was given as part of [TOB-TEESOLVER-4](#).

6. Race condition in create_liquidity_pool

Severity: Low

Difficulty: High

Type: Timing

Finding ID: TOB-TEESOLVER-6

Target: contracts/solver-registry/src/pool.rs

Description

When `call_result.is_err()` returns true, `create_liquidity_pool` can appear to succeed when, in reality, it fails.

The problem can be observed with a slight modification of the `create-pool.sh` script (figure 6.1). Note that the modified script contains two calls to `create_liquidity_pool` and that they run in parallel. Running the modified script produces output like in figure 6.2, showing that only one call succeeds.

```
near call $SOLVER_REGISTRY_CONTRACT create_liquidity_pool
'{"token_ids":["'$WNEAR_TOKEN',"'$USDC_TOKEN'], "fee":100}' --accountId
$SOLVER_TESTER_ACCOUNT_A --deposit 1.5 --gas 300000000000000 || echo 'first call
failed' &
near call $SOLVER_REGISTRY_CONTRACT create_liquidity_pool
'{"token_ids":["'$WNEAR_TOKEN',"'$USDC_TOKEN'], "fee":100}' --accountId
$SOLVER_TESTER_ACCOUNT_B --deposit 1.5 --gas 300000000000000 || echo 'second call
failed' &
```

Figure 6.1: Modification of the `create-pool.sh` script used to demonstrate the race condition

```
INFO Function execution logs -----
| Logs [...solver-registry...testnet]: No logs
| Logs [pool-0.solver-registry...testnet]: No logs
| Logs [...solver-registry...testnet]: No logs
| Logs [$SOLVER_TESTER_ACCOUNT_B]: No logs
| Logs [...solver-registry...testnet]: No logs
| -----
...
INFO Function execution logs -----
| Logs [...solver-registry...testnet]: No logs
| Logs [pool-0.solver-registry...testnet]: No logs
| Logs [$SOLVER_TESTER_ACCOUNT_A]:
|
EVENT_JSON:{"standard":"solver-registry", "version":"1.0.0", "event":"create_liquidity
_pool", "data":[{"pool_id":0, "token_ids":["wrap.testnet", "usdc.fakes.testnet"], "fee":
100}]}
```

Figure 6.2: Excerpt of the output produced by running the script in figure 6.1

One way to address this problem is to have `on_create_liquidity_pool` panic when `call_result.is_err()` returns true. Such a fix appears in figure 6.3.

```
110 pub fn on_create_liquidity_pool_account(  
111     &mut self,  
112     pool_id: u32,  
113     token_ids: Vec<AccountId>,  
114     fee: u32,  
115     #[callback_result] call_result: Result<(), PromiseError>,  
116 ) -> Option<u32> {  
117     if call_result.is_err() {  
118         // None // Proposed change  
119         env::panic_str("Failed to create pool"); //  
120     } else {  
121         ...  
122     }  
123 }
```

Figure 6.3: Proposed fix for the `on_create_liquidity_pool_account` function
([tee-solver/contracts/solver-registry/src/pool.rs#110-134](#))

Note that `deposit_into_pool` and `register_new_public_key` may similarly have race conditions, though we were unable to verify this before the end of the audit. For example, like `on_create_liquidity_pool`, `on_worker_key_added` does not panic when `call_result.is_err()` returns true (figure 6.4).

```
234 pub fn on_worker_key_added(  
235     &mut self,  
236     worker_id: AccountId,  
237     pool_id: u32,  
238     public_key: PublicKey,  
239     compose_hash: String,  
240     checksum: String,  
241     #[callback_result] call_result: Result<(), PromiseError>,  
242 ) {  
243     if call_result.is_ok() {  
244         ...  
245     } else {  
246         //  
247         env::panic_str("Failed to add worker key"); // Proposed change  
248     }  
249 }
```

Figure 6.4: Proposed fix for the potential race condition in `register_new_public_key`
([tee-solver/contracts/solver-registry/src/lib.rs#234-269](#))

Exploit Scenario

Alice and Bob both try to create liquidity pools at the same time. Both of their calls appear to succeed; however, only Alice's call was successful. Bob wastes time and energy trying to understand why he cannot administer the newly created pool.

Recommendations

Short term, take the following steps:

- Apply the fix in figure 6.3. This will ensure a user's call to `create_liquidity_pool` fails if it races with another user's call.
- Have the `deposit_into_pool` and `register_new_public_key` functions' callbacks panic when their passed-in callback result is an error. Thoroughly test the code following these changes. Applying these changes will communicate to the environment that the call has failed.

Long term, take the following steps:

- Test under adversarial conditions, such as multiple users trying to execute the same methods simultaneously.
- Test with `testnet` in addition to the sandbox environment provided by `near_workspaces`. (Note that we were not able to demonstrate the race condition using `testnet` as provided by `near_workspaces`.)

Taking these steps will help to expose problems like the one described in this finding.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Non-Security-Related Recommendations

This appendix contains recommendations that do not have immediate or obvious security implications. However, implementing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Enable Clippy's `missing-panics-doc` lint and address all of the warnings it generates.** Very few functions return `Results`. Hence, it is difficult to determine the functions that can fail and why they can fail.
- **Conspicuously mark the `LocalAttestation` type as “for testing purposes only.”** Doing so will reduce the likelihood that the type is accidentally used in production code.

```
41 pub enum Attestation {
42     Dstack(DstackAttestation),
43     Local(LocalAttestation),
44 }
45 ...
87 pub struct LocalAttestation {
88     verification_result: bool,
89 }
```

Figure B.1: Definition and use of `LocalAttestation`

([tee-solver/contracts/solver-registry/src/attestation/attestation.rs#41-89](#))

- **In tests, do not just verify that an operation failed; verify that it failed for the expected reason.** Checks like in figure B.2 appear in many places. However, the operation that produced the result might have failed for a reason other than the stated, expected one.

```
111 // Registration should fail with invalid TEE data
112 assert!(
113     !result.is_success(),
114     "Worker registration should fail with invalid quote hex"
115 );
```

Figure B.2: Pattern for checking results that appears in many tests

([tee-solver/contracts/solver-registry/tests/test_register_worker.rs#111-115](#))

- **Either remove the `shares` and `shares_total_supply` fields shown in figure B.3, or add a comment indicating that the fields are unused.**

```
26 /// Shares of the pool by liquidity providers.
```



```

27 pub shares: LookupMap<AccountId, Balance>,
28 /// Total number of shares.
29 pub shares_total_supply: Balance,

```

Figure B.3: Unused fields in `Pool` struct
([tee-solver/contracts/solver-registry/src/pool.rs#26-29](#))

- **Validate a pool's token IDs before the `on_create_liquidity_pool_account` callback is called.** Currently, the token IDs are validated in the callback's call to `Pool::new` (figures B.4 and B.5), which wastes gas.

```

110 pub fn on_create_liquidity_pool_account(
111     &mut self,
112     pool_id: u32,
113     token_ids: Vec<AccountId>,
114     fee: u32,
115     #[callback_result] call_result: Result<(), PromiseError>,
116 ) -> Option<u32> {
117     if call_result.is_err() {
118         None
119     } else {
120         // Add the new liquidity pool
121         let pool = Pool::new(token_ids.clone(), fee);

```

Figure B.4: Call to `Pool::new` in `on_create_liquidity_pool_account`
([tee-solver/contracts/solver-registry/src/pool.rs#110-121](#))

```

52 impl Pool {
53     pub fn new(token_ids: Vec<AccountId>, fee: u32) -> Self {
54         require!(token_ids.len() == 2, "Must have exactly 2 tokens");
55         require!(
56             token_ids[0] != token_ids[1],
57             "The two tokens cannot be identical"
58         );
59         require!(fee < 10_000, "Fee must be less than 100%");

```

Figure B.5: Token ID validation in `Pool::new`
([tee-solver/contracts/solver-registry/src/pool.rs#52-59](#))

- **Update the `deploy.sh` script to set the `worker_ping_timeout_ms` field during initialization.** Currently, running the script produces the error shown in figure B.6.

```

Error:
  0: Error: An error occurred during a `FunctionCall` action.
    ExecutionError("Smart contract panicked: Failed to deserialize input from
JSON. Error: `missing field `worker_ping_timeout_ms` at line 1 column 94`")

```

Figure B.6: Error message produced when the `deploy.sh` script is run

- **Update the `worker.sh` script to use the correct method and argument names (figure B.7).** `codehash` was renamed to `compose_hash`.

```

6    # approve worker codehash
7    near call $SOLVER_REGISTRY_CONTRACT approve_codehash
'{"codehash":"' $WORKER_CODEHASH '"}' --accountId $SOLVER_GOV_ACCOUNT

```

Figure B.7: Script with incorrect method and argument names
([tee-solver/scripts/testnet/worker.sh#6-7](#))

- Correct the typo in figure B.8 (“reseve” should be “reserve”).

```

73    > replace `token1.near`, `reseve_amount_1`, and `solver1.near` with your
actual values below:

```

Figure B.8: Typo in the AMM Solver’s README (“reseve” should be “reserve”)
([near-intents-tee-amm-solver/README.md#73](#))

- Correct the malformed @param tag in figure B.9 by adding names and descriptions.

```

106    /**
107    * Secure view function by cross-checking results from multiple NEAR RPC
nodes
108    * @param { contractId: string, methodName: string, args: object | undefined
}
109    * @returns validated view function result
110    */

```

Figure B.9: Malformed @param tag preceding secureViewFunction
([near-intents-tee-amm-solver/src/services/near.service.ts#106-110](#))

- Check named accounts in the .env file against NEAR_NETWORK_ID. For example, running npm start with the configuration in figure B.10 produces the error message in figure B.11. A more helpful error message would indicate that the SOLVER_REGISTRY_CONTRACT named account was for the wrong network.

```

NEAR_NETWORK_ID=mainnet
SOLVER_REGISTRY_CONTRACT=solver-registry-dev.testnet

```

Figure B.10: Sample configuration for the AMM Solver. Note that SOLVER_REGISTRY_CONTRACT is a testnet named account, not a mainnet named account.

```

TypeError: Can't complete the action because account solver-registry-dev.testnet
doesn't exist

```

Figure B.11: Error produced with the configuration in figure B.10

C. Steps to Reproduce TOB-TEESOLVER-4

This appendix contains steps to reproduce **TOB-TEESOLVER-4**: Invalid data returned from dstack SDK would cause AMM Solver to crash.

1. Install dstack per the instructions in the [dstackai/dstack](#) repository.⁴ We tested with version **0.19.28**.
2. Start the dstack server with the command `dstack server`.
3. Copy the configuration in figure C.1 to the file `env/.env.local`.
4. Run the command `NODE_ENV=local npm start`.

Following the last step, you should see the error depicted in figure 4.2 of **TOB-TEESOLVER-4**.

```
NEAR_NETWORK_ID=testnet

TEE_ENABLED=true
SOLVER_REGISTRY_CONTRACT=solver-registry-stg.testnet
SOLVER_POOL_ID=0

AMM_TOKEN1_ID=wrap.testnet
AMM_TOKEN2_ID=usdc.fakes.testnet

INTENTS_CONTRACT=mock-intents-2.testnet

# APP_PORT=3000
# LOG_LEVEL=info
# NEAR_NODE_URL=
NEAR_NODE_URLS=https://test.rpc.fastnear.com,https://neart.lava.build
# RELAY_WS_URL=wss://solver-relay-v2.chaindefuser.com/ws
# RELAY_AUTH_KEY=
MARGIN_PERCENT=0.3

DSTACK_SIMULATOR_ENDPOINT=http://127.0.0.1:3000
```

*Figure C.1: Configuration used to reproduce **TOB-TEESOLVER-4***

⁴ As explained in **TOB-TEESOLVER-4**, this version of dstack is not the one intended for use with the AMM Solver.

D. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

On January 15, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the NEAR One team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, of the six issues described in this report, NEAR One has resolved five issues and has not resolved the remaining issue. For additional information, please see the [Detailed Fix Review Results](#) below.

ID	Title	Severity	Status
1	Lack of two-step process for ownership transfers	Informational	Resolved
2	Code duplication	Informational	Unresolved
3	Insufficient test coverage	Informational	Resolved
4	Invalid data returned from dstack SDK would cause AMM Solver to crash	Low	Resolved
5	Lack of an mt_batch_balance_of method causes AMM Solver to crash	Low	Resolved
6	Race condition in create_liquidity_pool	Low	Resolved

Detailed Fix Review Results

TOB-TEESOLVER-1: Lack of two-step process for ownership transfers

Resolved by [commit 3db2244](#). The `change_owner` function was removed. Hence, the lack of a two-step process no longer applies.

TOB-TEESOLVER-2: Code duplication

Unresolved. The client provided the following context for this finding's fix status:

We're working on a TEE attestation crate that will serve the goal, but it's not yet in this version.

TOB-TEESOLVER-3: Insufficient test coverage

Resolved by [commit d518d5f](#) of [PR #16](#). As stated in the PR, two of the named functions (`change_owner` and `get_owner_id`) were removed and thus can no longer be tested. Also, the error case in `find_approved_compose_hash` is unreachable because of a `require!` statement that precedes the error case. For the remaining 12 locations, we verified that each is now exercised by a test.

Note that we were unable to run the tests locally on either macOS Tahoe 26.2 or Ubuntu 24.04.3. Furthermore, we were unable to determine the reason for the difficulty. Ultimately, we tested the fixes in GitHub Codespaces using Ubuntu 24.04.3.

TOB-TEESOLVER-4: Invalid data returned from dstack SDK would cause AMM Solver to crash

Resolved by [commit f6a5e67](#) of [PR #10](#). The AMM Solver no longer crashes with the message given in figure 4.2. The error is now caught, and the error in figure D.1 is emitted.

```
.../src/utils/agent.ts:157
    throw new Error(`NOT running in TEE or failed to get TEE quote or collateral:
    ${errorMessage}`);
      ^
Error: NOT running in TEE or failed to get TEE quote or collateral: Cannot read
properties of undefined (reading 'replace')
```

Figure D.1: New error message produced when an invalid quote is returned to the AMM Solver

TOB-TEESOLVER-5: Lack of an `mt_batch_balance_of` method causes AMM Solver to crash

Resolved by [commit 75ae11a](#) of [PR #10](#). The AMM Solver no longer crashes with the message given in figure 5.2. The error is now caught, and the error in figure D.2 is emitted.

```
.../src/services/intents.service.ts:24
    throw new Error(`Failed to validate reserves on NEAR Intents contract
    ${intentsContract}. ${errorMessage}`);
      ^
```

```
Error: Failed to validate reserves on NEAR Intents contract mock-intents-2.testnet.  
Contract method is not found
```

Figure D.2: New error message produced when the `mt_batch_balance_of` method cannot be found in the intents contract

TOB-TEESOLVER-6: Race condition in `create_liquidity_pool`

Resolved by [commit e7ca820](#) of [PR #15](#). Repeating the experiment in figure 6.1 no longer produces the output in figure 6.2. Now, the output in figure D.3 is produced. Note the “Failed to create liquidity pool account” message, which was introduced by [commit 10d3d5e](#) of [PR #15](#) (figure D.4).

```
ERROR Transaction failed  
|   Gas burned: 20.6 Tgas  
|   Transaction fee: 0.0020074075479994 NEAR  
|   Transaction ID: 3YxCR5hyT5VRwhkTpJjQUzFLNpkESeJpH8T3jQRX7BY8  
|   To see the transaction in the transaction explorer, please open this url in  
your browser:  
|  
https://explorer.testnet.near.org/transactions/3YxCR5hyT5VRwhkTpJjQUzFLNpkESeJpH8T3j  
QRX7BY8  
  
▶▶▶▶ Sending transaction ...  
Here is your console command if you need to script it or re-run:  
    near contract call-function as-transaction solver-registr-test-2026-0117.testnet  
create_liquidity_pool json-args  
'{"token_ids":["wrap.testnet","usdc.fakes.testnet"],"fee":100}' prepaid-gas '300.0  
Tgas' attached-deposit '1.5 NEAR' sign-as account-sam-4.testnet network-config  
testnet sign-with-keychain send  
  
`near` CLI has a new update available 0.20.0 → 0.23.5  
To update `near` CLI use: near extensions self-update  
Error:  
    0: Error: An error occurred during a `FunctionCall` action.  
        ExecutionError("Smart contract panicked: Failed to create liquidity pool  
account")
```

Figure D.3: New error message produced when a call to `create_liquidity_pool` fails

```

124 pub fn on_create_liquidity_pool_account(
125     &mut self,
126     pool_id: u32,
127     token_ids: &Vec<AccountId>,
128     fee: u32,
129     #[callback_result] call_result: Result<(), PromiseError>,
130 ) -> u32 {
131     if call_result.is_ok() {
132         ...
133     } else {
134         env::panic_str("Failed to create liquidity pool account");
135     }
136 }

```

*Figure D.4: Error message introduced by PR #15
(tee-solver/contracts/solver-registry/src/pool.rs#124-148)*

Please also note that reproducing the output in figure 6.2 of **TOB-TEESOLVER-6** requires near-cli-rs 0.20.0.

E. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	Description
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2025 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to NEAR One under the terms of the project statement of work and has been made public at NEAR One's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.